Code-Extrakt: Spotify Artist-Profil öffnen

Dieser Extrakt enthält alle Code-Bausteine, die zusammen das Öffnen eines validierten Spotify-Künstlerprofils ermöglichen – vom serverseitigen Token-Abruf bis zum Deep-Link im Browser.

---

1. Serverseitig: Token-Caching (`server/spotify.ts`)

Der Token wird einmalig geholt und im Speicher gecacht. Erst wenn er abläuft (minus 60 s Puffer), wird ein neuer angefordert.

```typescript
// In-Memory Token Cache
let tokenCache: { access_token: string; expires_at: number } | null = null;

export async function getSpotifyToken(): Promise<string> {
  const now = Date.now();

  // Cache-Hit: Token noch mindestens 60 Sekunden gültig
  if (tokenCache && tokenCache.expires_at > now + 60_000) {
    return tokenCache.access_token;
  }

  const credentials = Buffer.from(
    `${process.env.SPOTIFY_CLIENT_ID}:${process.env.SPOTIFY_CLIENT_SECRET}`
  ).toString("base64");

  const response = await fetch("https://accounts.spotify.com/api/token", {
    method: "POST",
    headers: {
      Authorization: `Basic ${credentials}`,
      "Content-Type": "application/x-www-form-urlencoded",
    },
    body: "grant_type=client_credentials",
  });

  const data = await response.json();
  tokenCache = {
    access_token: data.access_token,
    expires_at: now + data.expires_in * 1000,
  };
  return tokenCache.access_token;
}
```

---

2. Serverseitig: Künstler suchen & validieren (`server/spotify.ts`)

Nimmt einen Rohnamen entgegen, gibt ein validiertes Objekt mit `spotify_id` und `direct_link` zurück – oder `null` bei keinem Treffer.

```typescript
export async function searchSpotifyArtist(
  artistName: string
): Promise<SpotifyArtistResult | null> {
  const token = await getSpotifyToken();

  const params = new URLSearchParams({
    q: artistName,   // URL-encoded Künstlername
    type: "artist",
    limit: "1",      // Nur relevantester Treffer
  });

  const response = await fetch(
    `https://api.spotify.com/v1/search?${params}`,
    { headers: { Authorization: `Bearer ${token}` } }
  );

  const data = await response.json();
  const artist = data.artists?.items?.[0];
  if (!artist) return null; // Kein Treffer → kein kaputter Link

  return {
    display_name: artistName,              // Originale Sucheingabe
    spotify_name: artist.name,             // Offizieller Spotify-Name
    spotify_id: artist.id,                 // z.B. "4gzpq5YpGjS9uS06r8Iu0S"
    direct_link: artist.external_urls.spotify, // Deep-Link zur nativen App
    image_url: artist.images[0]?.url ?? null,
    genres: artist.genres,
    followers: artist.followers.total,
    popularity: artist.popularity,
  };
}
```

Zurückgegebenes JSON-Objekt:

```json
{
  "display_name": "Coldplay",
  "spotify_name": "Coldplay",
  "spotify_id": "4gzpq5YpGjS9uS06r8Iu0S",
  "direct_link": "https://open.spotify.com/artist/4gzpq5YpGjS9uS06r8Iu0S",
  "image_url": "https://i.scdn.co/image/...",
  "genres": ["pop", "post-grunge"],
  "followers": 35000000,
  "popularity": 88
}
```

---

3. Serverseitig: tRPC-Prozedur (`server/routers/artist.ts`)

Die Prozedur prüft zuerst den DB-Cache, ruft bei Cache-Miss die Spotify API auf und speichert das Ergebnis.

```
search: publicProcedure
  .input(z.object({ query: z.string().min(1).max(200) }))
  .mutation(async ({ input }) => {
    // 1. DB-Cache prüfen
    const cached = await searchCachedArtists(input.query);
    if (cached.length > 0) {
      return { found: true, artist: { ...cached[0], genres:
JSON.parse(cached[0].genres) }, fromCache: true };
    }

    // 2. Spotify API aufrufen
    const result = await searchSpotifyArtist(input.query);
    if (!result) return { found: false, artist: null, fromCache: false };

    // 3. Ergebnis in DB cachen
    await upsertArtist({
      spotifyId: result.spotify_id,
      spotifyName: result.spotify_name,
      directLink: result.direct_link,
      imageUrl: result.image_url,
      genres: JSON.stringify(result.genres),
      followers: result.followers,
      popularity: result.popularity,
    });

    return { found: true, artist: { ...result }, fromCache: false };
  }),
```

---

4. Frontend: Deep-Link `<a>`-Tag (`client/src/components/ArtistCard.tsx`)

Das ist der kritische Teil: `target="_blank"` + `rel="noopener noreferrer"` umgeht die iframe-Sandbox und triggert die native Spotify-App auf Mobilgeräten.

```
{directLink ? (
  // ✅ Deep-Link – öffnet native Spotify-App auf Mobilgeräten
  <a
    href={directLink}
    target="_blank"
    rel="noopener noreferrer"
    aria-label={`${spotifyName} auf Spotify öffnen`}
  >
    Auf Spotify öffnen
  </a>
) : (
  // ✅ Fallback – kein kaputter Link, nur neutraler Text
  <span>Kein Link verfügbar</span>
)}
```

---

5. Frontend: Suche auslösen & Ergebnis anzeigen (`client/src/pages/Home.tsx`)

```
const searchMutation = trpc.artist.search.useMutation({
  onSuccess: (data) => {
    setSearchResult(data);
  },
});

// Suche starten
searchMutation.mutate({ query: "Coldplay" });

// Ergebnis rendern
{searchResult?.found && searchResult.artist && (
  <a
    href={searchResult.artist.directLink}
    target="_blank"
    rel="noopener noreferrer"
  >
    {searchResult.artist.spotifyName} auf Spotify öffnen
  </a>
)}
```

Zusammenfassung: Datenfluss

```
Nutzer gibt Namen ein
        ↓
trpc.artist.search.mutate({ query })
        ↓
[Server] DB-Cache prüfen → Cache-Hit? → sofort zurückgeben
        ↓ (Cache-Miss)
GET https://api.spotify.com/v1/search?q={name}&type=artist&limit=1
  Header: Authorization: Bearer {access_token}
        ↓
artist.external_urls.spotify  →  "https://open.spotify.com/artist/{id}"
        ↓
<a href="..." target="_blank" rel="noopener noreferrer">
  → öffnet native Spotify-App auf Mobilgerät
```